

Topic 8: Sound Transformations

Spectral filtering, scaling and morphing with STFT and sound models

Digital Signal Processing for Sound and Music · Topic 8

Xavier Serra · Universitat Pompeu Fabra

Lecture roadmap

Representations, transformations, examples and Exercise 8

1 STFT

Filtering and spectral
morphing

2 Sinusoidal tracks

Frequency, magnitude and
time

3 Harmonic structure

Transposition, shifting and
stretching

4 HPS representation

Harmonics and stochastic
envelopes

5 Morphing

Separate controls for
model components

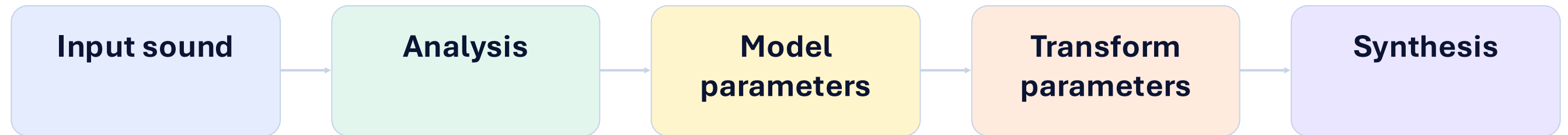
6 Examples + Lab 8

Inspect, listen and explain
parameter choices

Analysis, transformation and synthesis

The representation determines which sound properties we can control

- Analysis estimates a representation of the input sound.
- Transformation changes selected values in that representation.
- Synthesis generates a waveform from the modified values.



The first comparison is always original sound versus unmodified resynthesis.

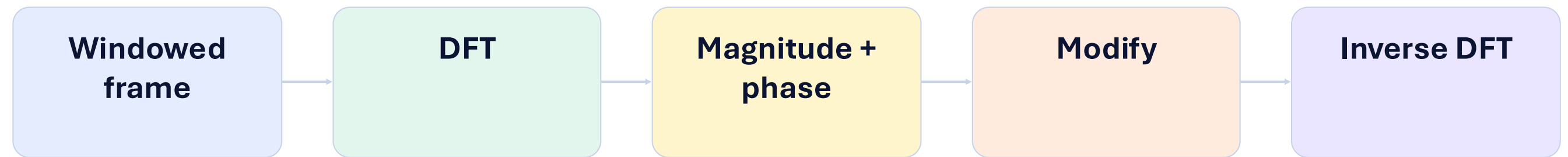
Representations and their controls

Increasing abstraction makes some transformations simpler and removes some detail

Representation	Stored information	Useful controls
STFT	Magnitude and phase in every bin	Spectral filtering, envelope morphing
Sinusoidal	Frequency, magnitude and phase tracks	Move components and retime tracks
Harmonic	Tracks organized by harmonic number	Pitch, inharmonicity, spectral envelope
HPR	Harmonics plus the complex residual	Transform harmonics while retaining residual
HPS	Harmonics plus a smooth noise envelope	Separate tonal and noise transformation

STFT transformations

Modify each windowed spectrum, then reconstruct overlapping frames



Reconstruction

- Overlap-add combines the inverse frames into a continuous sound.
- Compatible windows and gain normalization prevent periodic level modulation.
- After a modification, the spectra may no longer correspond exactly to one consistent STFT.

Filtering with the STFT

A frequency response changes the magnitude and, for a complex filter, the phase

Complex spectral multiplication

$$Y_\ell[k] = H[k] X_\ell[k]$$

Magnitude in dB

$$mY_\ell[k] = mX_\ell[k] + G[k]$$

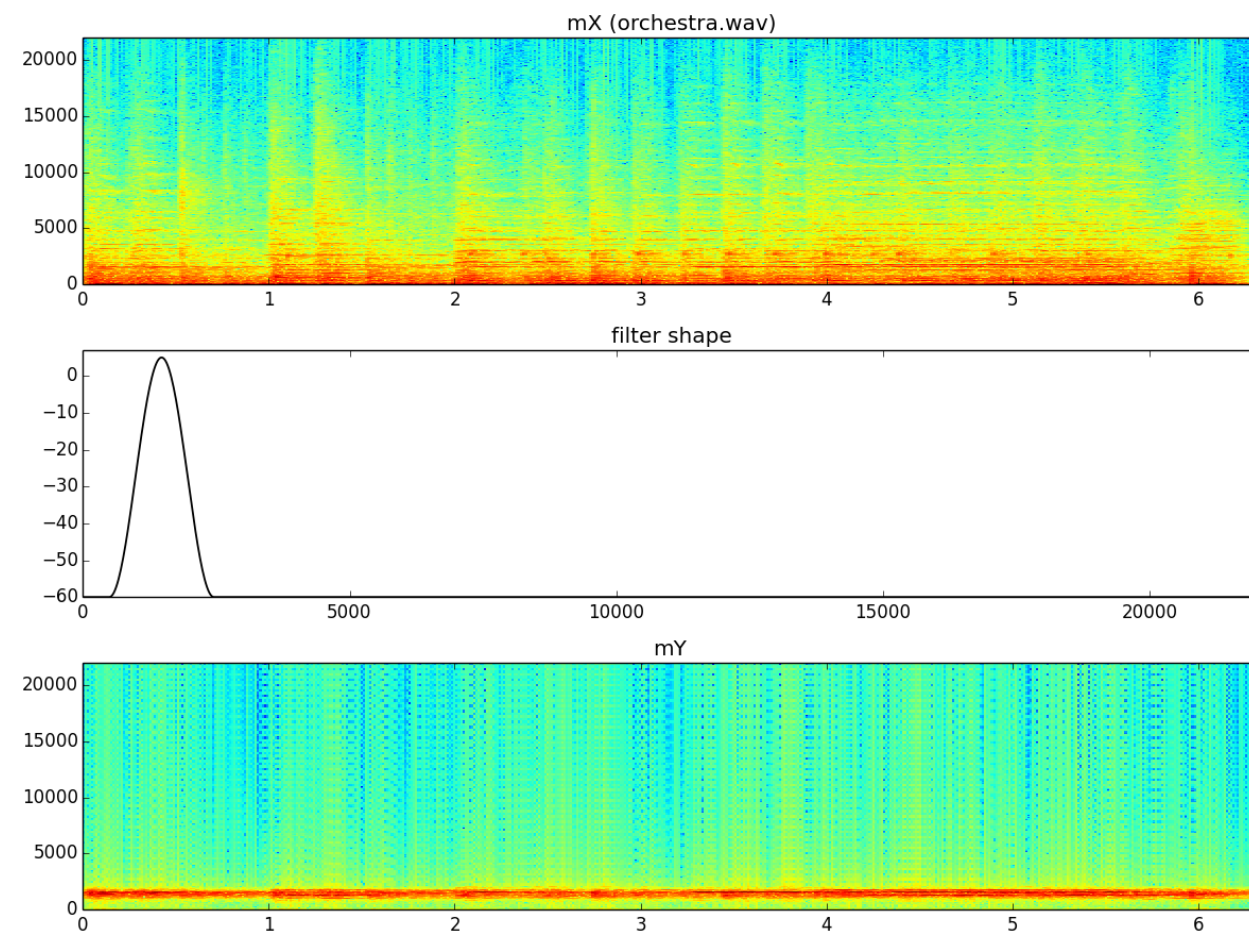
$$\varphi Y_\ell[k] = \varphi X_\ell[k] + \arg H[k]$$

How the code filters

- $G[k]$ is the filter gain in dB. Negative gain attenuates the band.
- `stftFiltering()` adds $G[k]$ to dB magnitudes and keeps the input phase.
- A smooth response avoids abrupt spectral changes.

Example: spectral filtering of an orchestra sound

Read the filter shape between the input and output spectrograms



Original example from 8T1: orchestra.wav

What changes?

- The filter retains a selected frequency band.
- Energy outside that band decreases.
- Musical timing remains aligned with the input.
- Listen for a narrower, altered timbre.

Morphing with STFT magnitude envelopes

The Python example interpolates dB magnitudes and retains the phase of sound 1

Spectral interpolation

$$mY_\ell[k] = (1 - \alpha) mX_{1,\ell}[k] + \alpha E_{2,\ell}[k]$$

$$\varphi Y_\ell[k] = \varphi X_{1,\ell}[k]$$

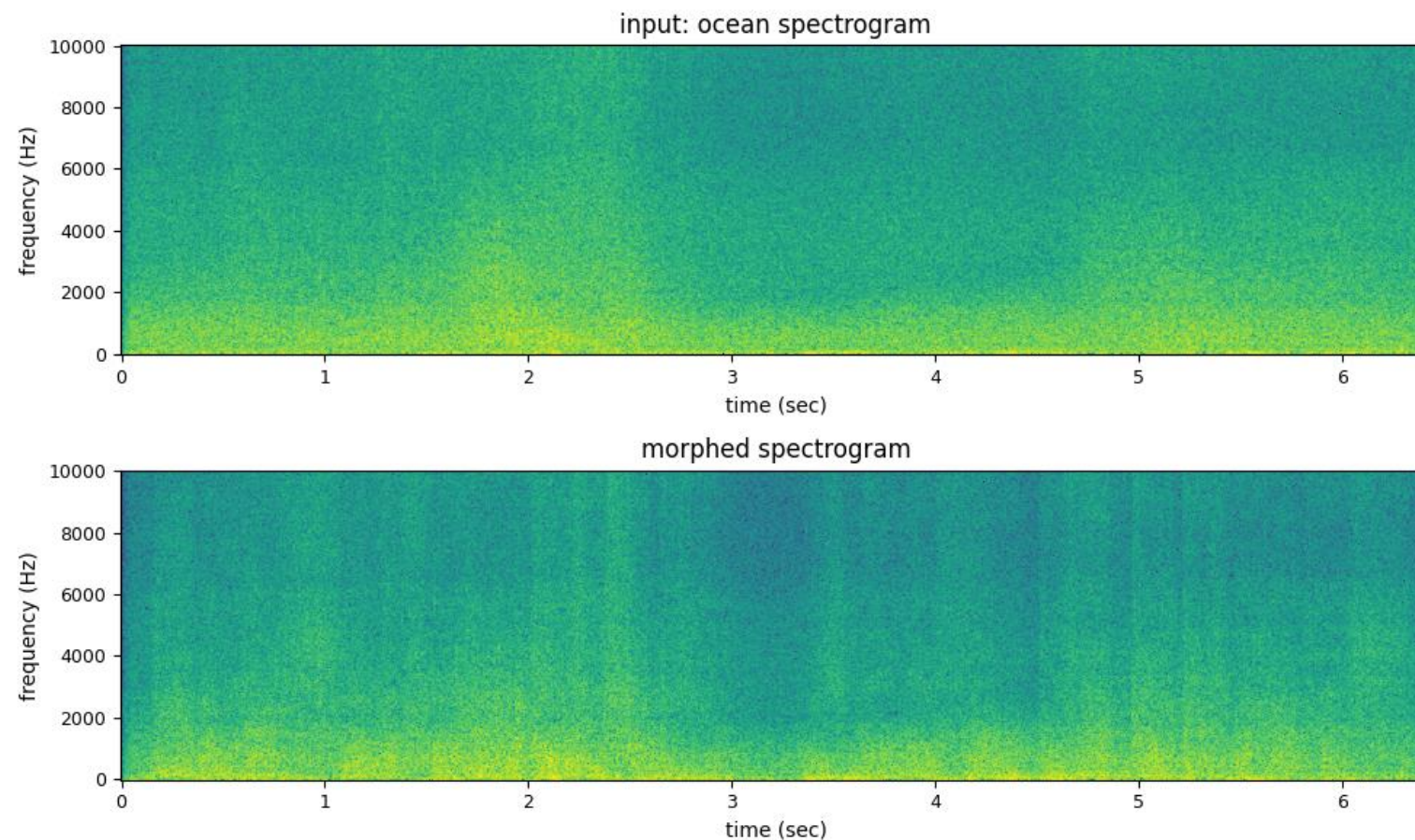
E2 is the smoothed dB magnitude of sound 2.
 α is balance_factor, between 0 and 1.

Controls in stftMorph()

- balance_factor controls how much of sound 2's magnitude envelope enters the result.
- A smaller smooth_factor keeps a coarser spectral envelope.
- At balance 1, the phase still comes from sound 1.

Example: ocean texture with a speech-shaped spectrum

The spectral envelope changes while the output keeps sound 1 as its phase reference



[stft-morph-examples.ipynb](#)

Example settings

- Inputs: ocean.wav and speech-male.wav.
- $M = N = 1024$ and $H = 256$.
- `smooth_factor = 0.5` and `balance_factor = 0.2`.
- Compare spectral shaping with intelligibility by listening.

Waveform crossfading and model morphing

Different interpolation domains produce different audible results

Waveform crossfade

$$y[n] = (1 - \alpha) x_1[n] + \alpha x_2[n]$$

- Mixes two existing waveforms.
- The listener can often hear both sources.
- Their phase relationships can create cancellation.

Model morph

- Interpolates spectral or model parameters.
- Can combine the pitch structure of one sound with the envelope of another.
- Requires compatible timing and parameter correspondence.

A hybrid sound depends on which parameters we interpolate and which we keep.

Transformations of sinusoidal tracks

Move frequencies, change magnitudes and remap time before synthesis

Output frame q reads input position $u(q)$

$$f'_r[q] = s_f[q] f_r(u(q))$$

$$m'_r[q] = m_r(u(q)) + g_{dB}[q]$$

r : track index q : output frame

f : frequency in Hz m : magnitude in dB

Meaning of each control

- Frequency scaling multiplies every active track frequency.
- Adding dB gain changes the track amplitude.
- The time map changes when a track value appears in the output.

Phase after a frequency or time transformation

Continuous phase must follow the new frequency trajectory

Sample-by-sample phase accumulation

$$\varphi_r[n] = \varphi_r[n - 1] + \frac{2\pi f_r[n]}{f_s}$$

Approximate phase advance over one hop

$$\Delta\varphi_r \approx \frac{2\pi H f_r}{f_s}$$

Why phases are regenerated

- Original phases describe the original frequencies and timing.
- Reusing them can create discontinuities.
- An empty phase array requests new phase evolution.

Time scaling and frequency scaling

Model parameters let duration and pitch change independently

Operation	Duration	Frequencies
Frequency scaling by 2	Approximately unchanged	Every active frequency doubles
Time scaling by 2	Approximately doubles	Track frequencies stay at their original values
Both operations	Approximately doubles	Every active frequency doubles
Slower sample playback	Increases	All frequencies decrease together

Frequency scaling changes the vertical track positions. Time scaling changes their timing.

Time-scaling arrays are input–output time maps

Each pair specifies where an input event should occur in the output

$$t_{out} = g(t_{in})$$

[0, 0, 1, 2]
(0, 0) and (1, 2)

- The input endpoint maps to twice its original duration.
- A slope above 1 stretches that segment.
- A slope below 1 compresses that segment.

A local example

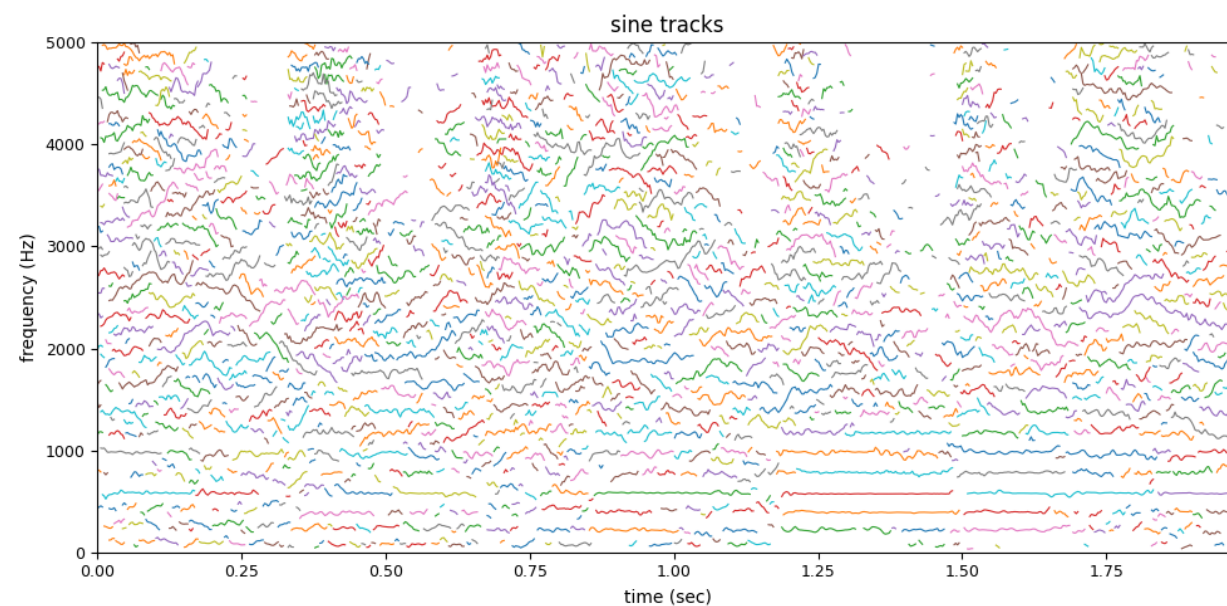
[0, 0, 0.5, 0.5, 1, 1.5]

- The first half keeps its duration.
- The second half occupies twice as much output time.
- Both time axes must increase. The coordinates can use seconds or a consistent normalized scale.

Example: sinusoidal transformation of mridangam

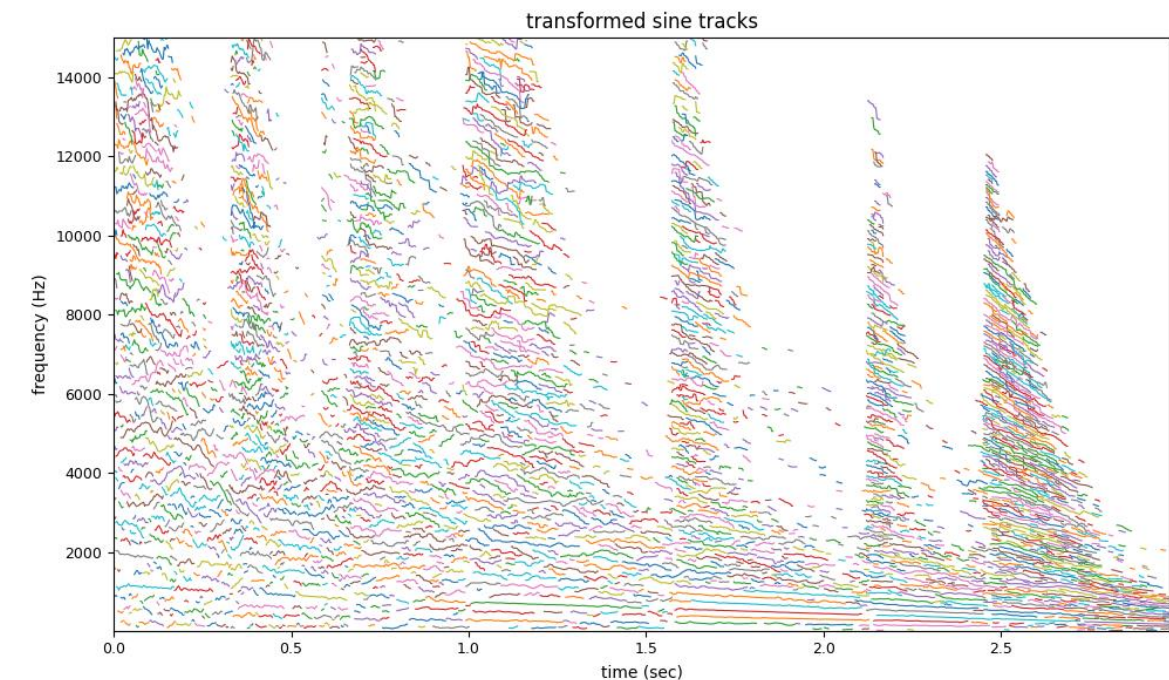
Frequency scaling moves the tracks while the time map changes the spacing of events

Original tracks



[sine-transformations-examples.ipynb](#)

Transformed tracks



Frequency factor evolves from 2.0 to 0.3. Time landmarks: 0, 0.671 and 1.978 s. The final landmark moves to 2.978 s. The two plots use different frequency-axis limits.

Harmonic plus residual transformations

Transform a harmonic component and recombine it with the remaining signal

Decomposition

$$x[n] = x_h[n] + r[n]$$

$$R_\ell[k] = X_\ell[k] - X_{h,\ell}[k]$$

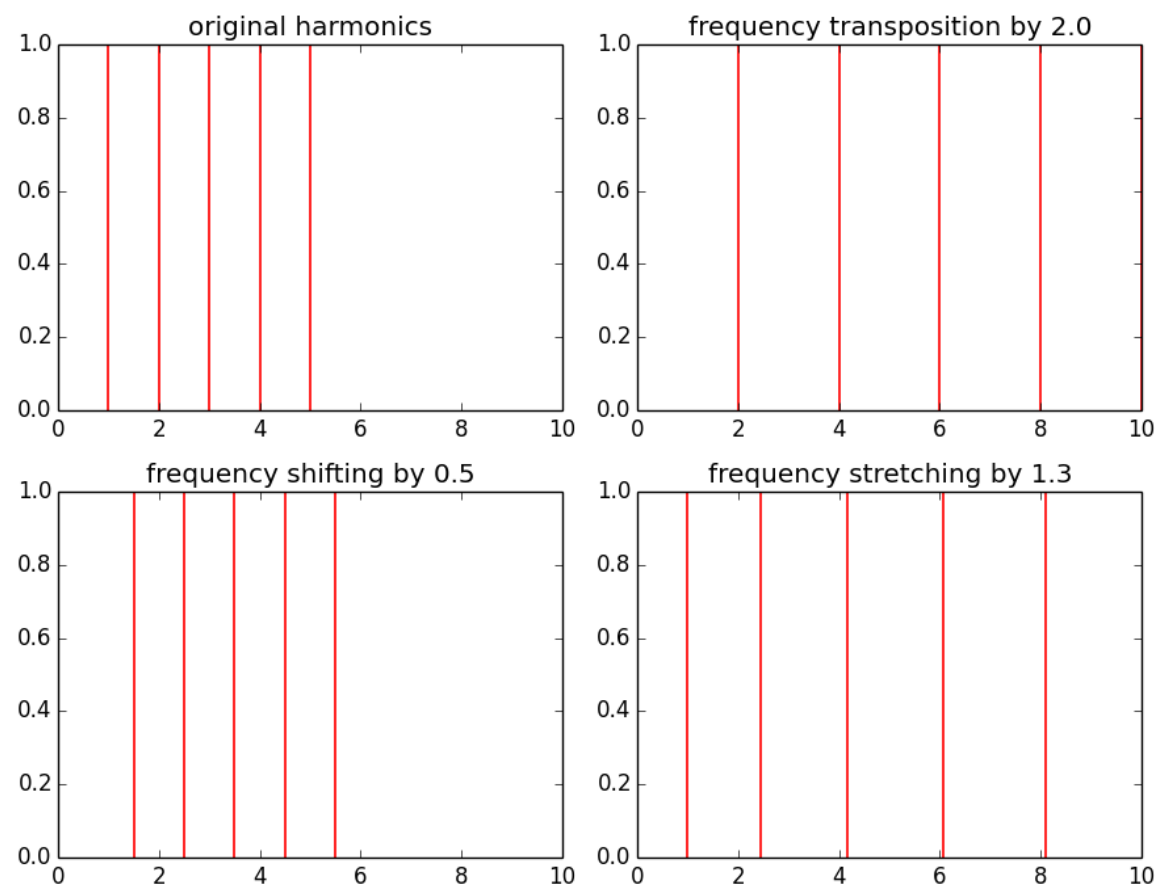
The residual retains everything not explained by the estimated harmonic model.

After the transformation

- Synthesize the modified harmonic tracks.
- Add the residual in a compatible representation and alignment.
- Leaving residual pitch cues unchanged can conflict with a large harmonic transposition.

Frequency transposition, shifting and stretching

The same set of partials changes differently under each transformation



Original 8T2 illustration: power-law stretching

Transposition

$$f'_h = a f_h$$

Frequency shifting

$$f'_h = f_h + d$$

Power-law stretching

$$f'_h = \frac{f_h}{h} h^\beta$$

h starts at 1.

a and β are dimensionless.

d is in Hz.

The stretching rule in the Python examples

The effect grows with harmonic number, while the first harmonic keeps its scaling factor

$$f'_h = a f_h b^{h-1}$$

Partial h	Original (Hz)	Scale a = 1.5	Shift d = 100 Hz	Stretch b = 1.2
1	200	300	300	200
2	400	600	500	480
3	600	900	700	864
4	800	1200	900	1382.4

For stretching: a = 1. Higher partials move progressively farther from a harmonic series.

Pitch transformation and spectral-envelope preservation

Harmonic spacing determines pitch, while the envelope strongly influences timbre

timbrePreservation = 0

- Move frequencies and retain each harmonic magnitude.
- Spectral-envelope features move with the partials.
- Speech can sound smaller or larger as pitch and formant structure move together.

timbrePreservation = 1

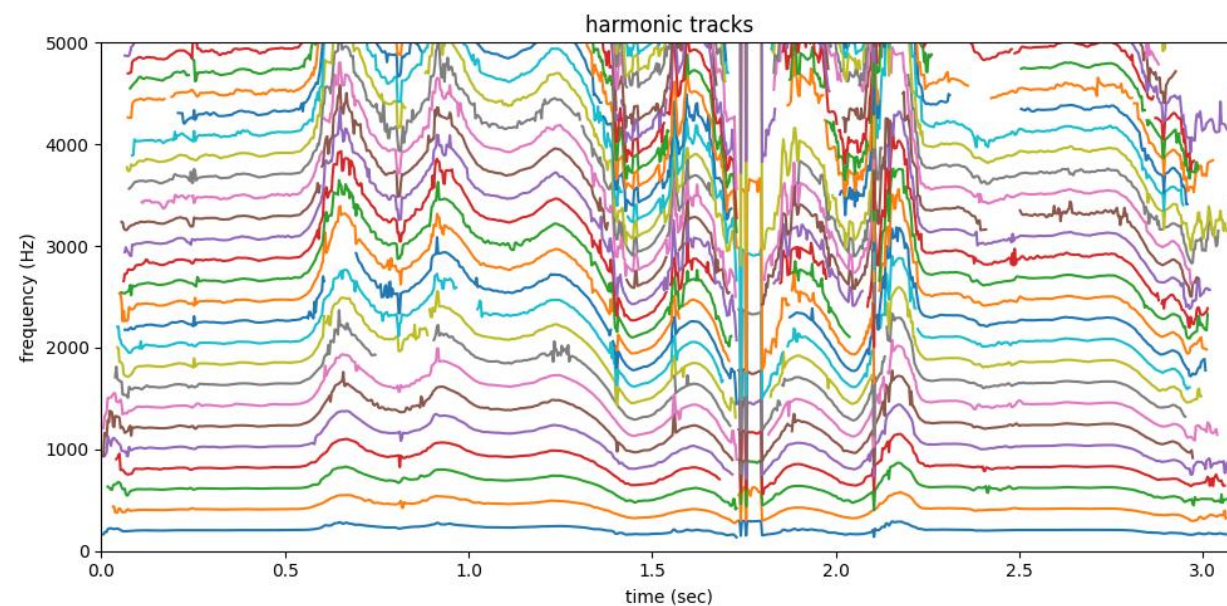
$$m'_h = E(f'_h)$$

- Estimate the original spectral envelope $E(f)$.
- Read a new magnitude at each transformed harmonic frequency.
- This approximates envelope preservation. It cannot guarantee the original speaker identity.

Example: harmonic transformation of a singing voice

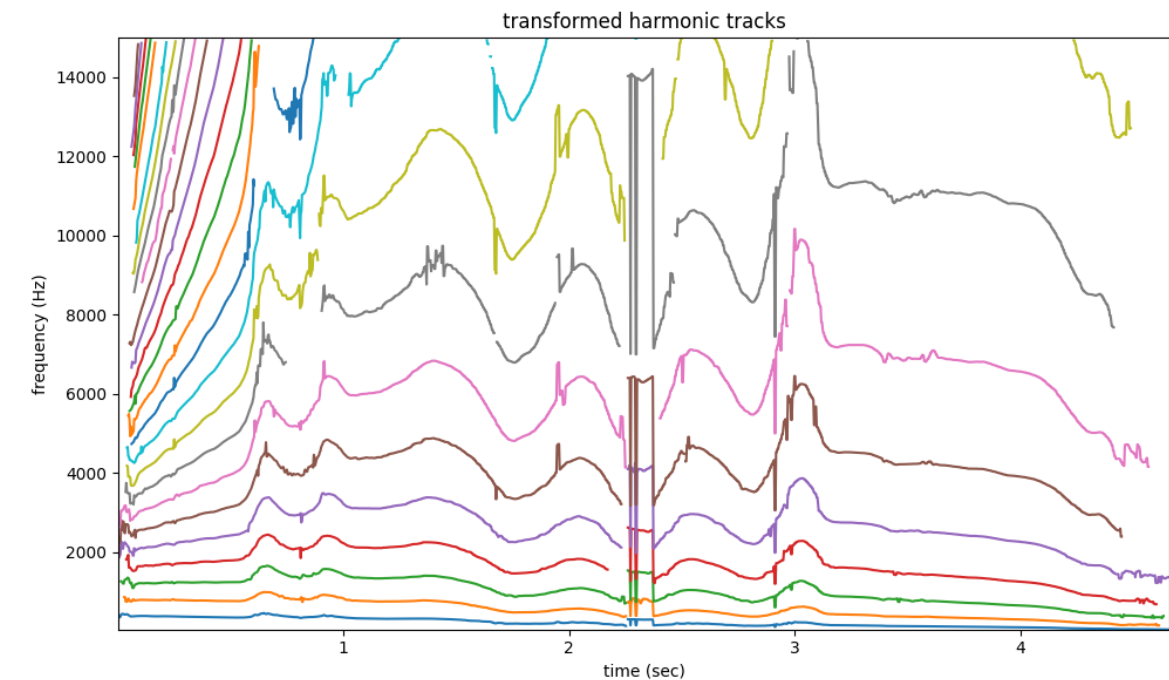
Scaling changes pitch and stretching bends the upper partials away from harmonicity

Original harmonics



[harmonic-transformations-examples.ipynb](#)

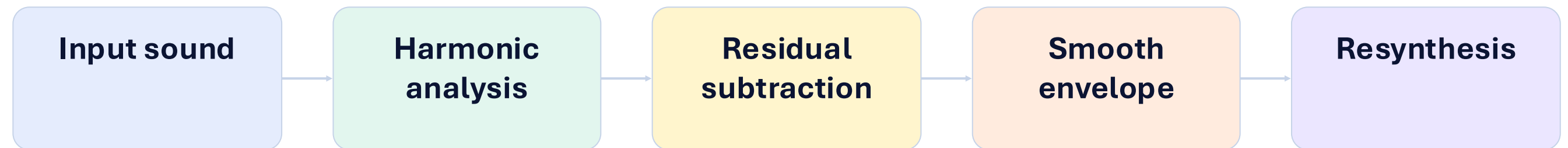
Transformed harmonics



Input: vignesh.wav. Scaling moves from 2.0 to 0.3. Stretching moves from 1 to 1.5. Envelope preservation is enabled. Compare spacing, timing and sound.

Harmonic plus stochastic representation

HPS replaces the detailed residual with a smooth spectral envelope and regenerated noise



$$y[n] = y_h[n] + y_{st}[n]$$

- Harmonic tracks describe stable pitched components.
- The stochastic envelope describes the evolving distribution of noise energy.
- Noise synthesis uses random phase, so its waveform differs between realizations.

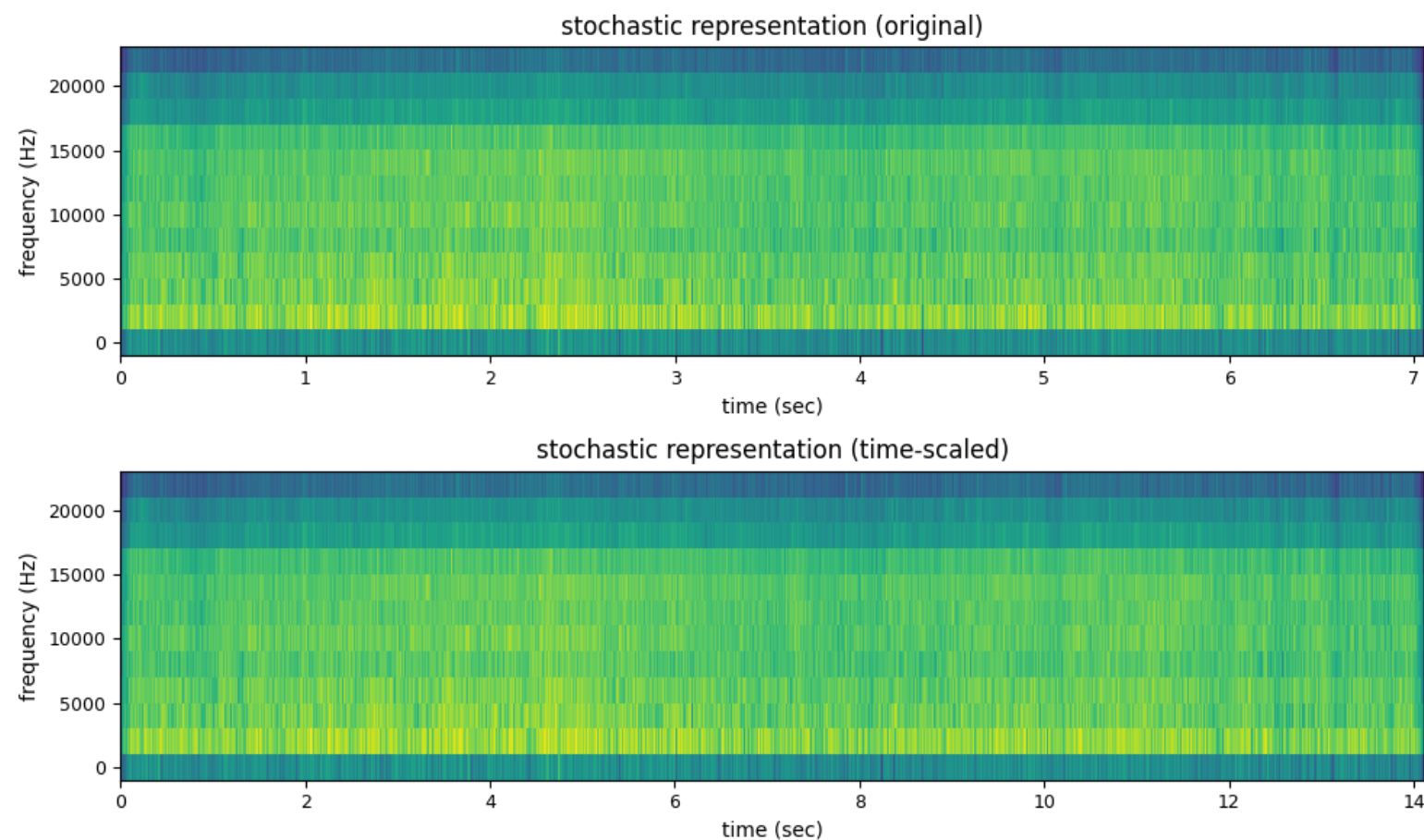
Harmonic plus residual and harmonic plus stochastic

The difference lies in what the model retains after subtracting the harmonics

Property	HPR	HPS
Non-harmonic component	Detailed residual spectrum or waveform	Smooth magnitude envelope
Phase of that component	Retained when residual stays unchanged	Random phase regenerated at synthesis
Fine temporal detail	Can preserve attacks and residual structure	May smooth attacks and fine structure
Transformation	Detailed residual is harder to retime	Compact envelope is easier to retime
Reconstruction goal	Retain the residual faithfully	Approximate its spectral and perceptual character

Example: time scaling a stochastic rain sound

Retiming the envelope changes texture duration while synthesis creates a new noise realization



[stochastic-transformations-examples.ipynb](#)

What to compare

- `stoc_factor = 0.1` keeps a compact envelope.
- The map `[0, 0, 1, 2]` doubles duration.
- Follow the horizontal timing of spectral regions.
- Waveform identity is not the aim of stochastic resynthesis.

Transformations of an HPS representation

Harmonics and stochastic envelopes should share the same time map

Harmonic control

$$f'_h[q] = a[q] f_h(u(q)) b[q]^{h-1}$$

Stochastic control

$$E'_q[k] = E_{u(q)}[k] + g_{noise}[q]$$

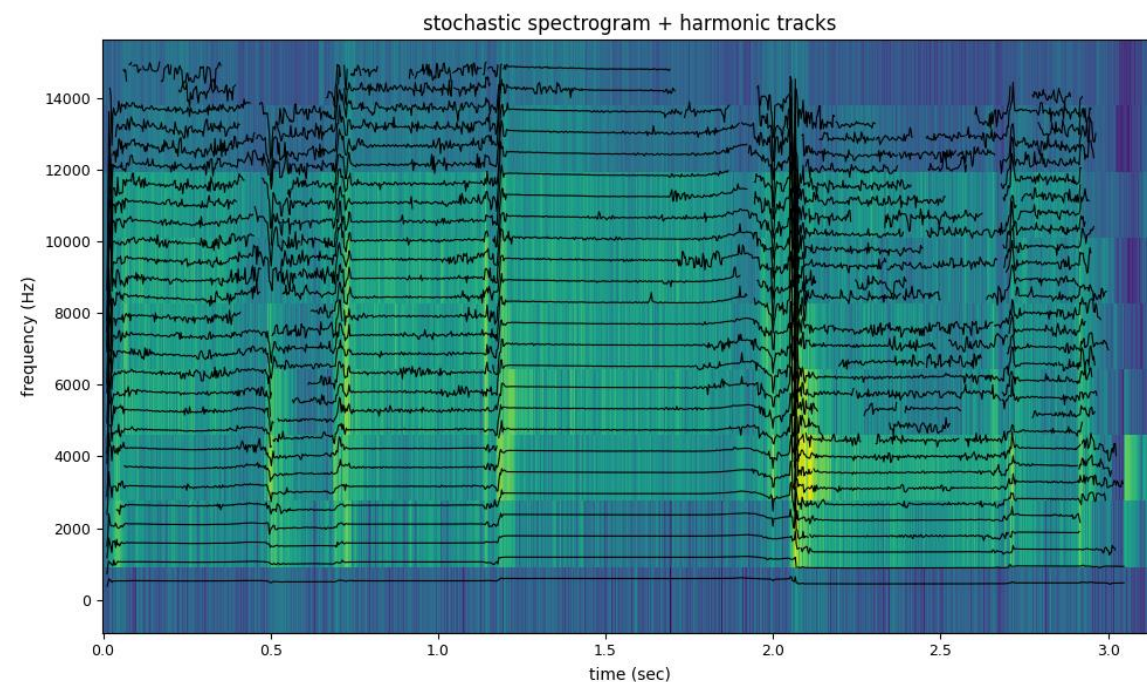
A coherent output

- Retiming both parts together keeps noisy and tonal events aligned.
- A dB envelope gain changes stochastic energy.
- The E8 frequency call changes the harmonics. It leaves the stochastic frequency envelope unchanged.

Example: HPS transformation of a saxophone phrase

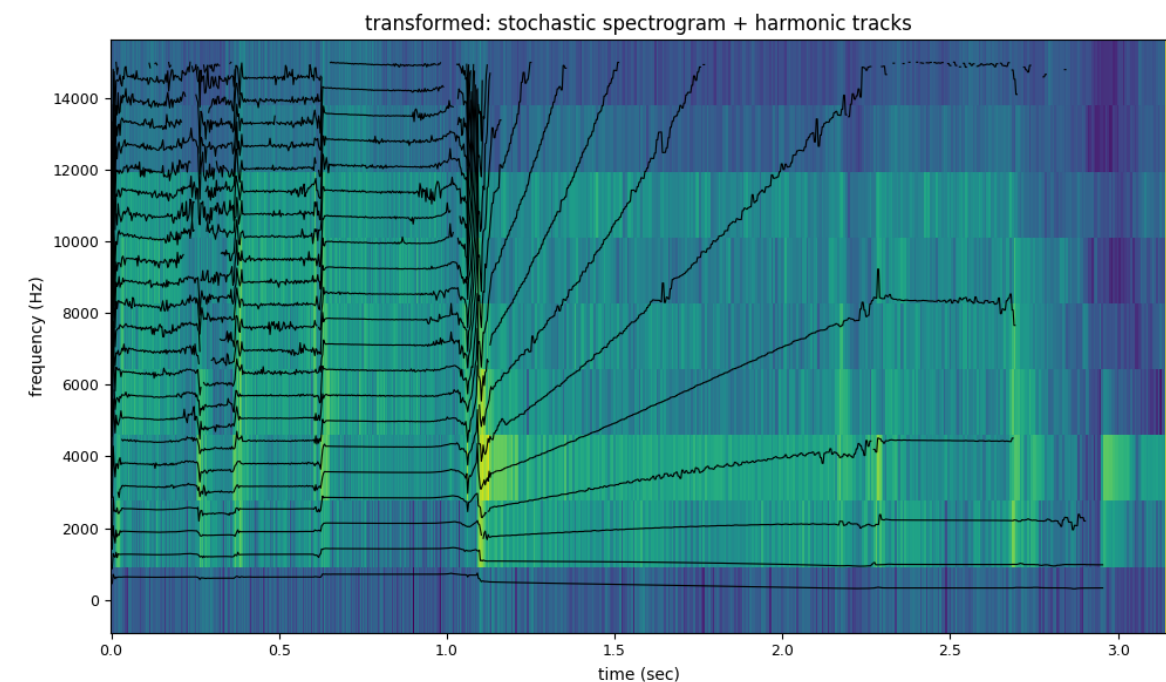
Read the harmonic tracks and the stochastic background as different parts of one sound

Original HPS



hps-transformations-examples.ipynb

Transformed HPS



Input: sax-phrase-short.wav. Black curves show harmonics. Color shows the stochastic envelope. Scaling evolves from 1.2 to 0.7. Later stretching reaches 1.5, with envelope preservation enabled.

Example: local timing changes in the saxophone phrase

A nonlinear time map can compress one region and stretch another without changing total duration

Input time (s)	Output time (s)	Meaning
0	0	Same start
2.138	1.138	Earlier region compresses
3.146	3.146	Later region stretches to the same final endpoint

time_scaling = [0, 0, 2.138, 1.138, 3.146, 3.146]

The overall duration stays approximately constant, but internal events move.

Morphing with harmonic plus stochastic models

Separate interpolation controls can create different combinations of pitch and timbre

$$f'_h = (1 - \alpha_f) f_{h,1} + \alpha_f f_{h,2}$$

$$m'_h = (1 - \alpha_m) m_{h,1} + \alpha_m m_{h,2}$$

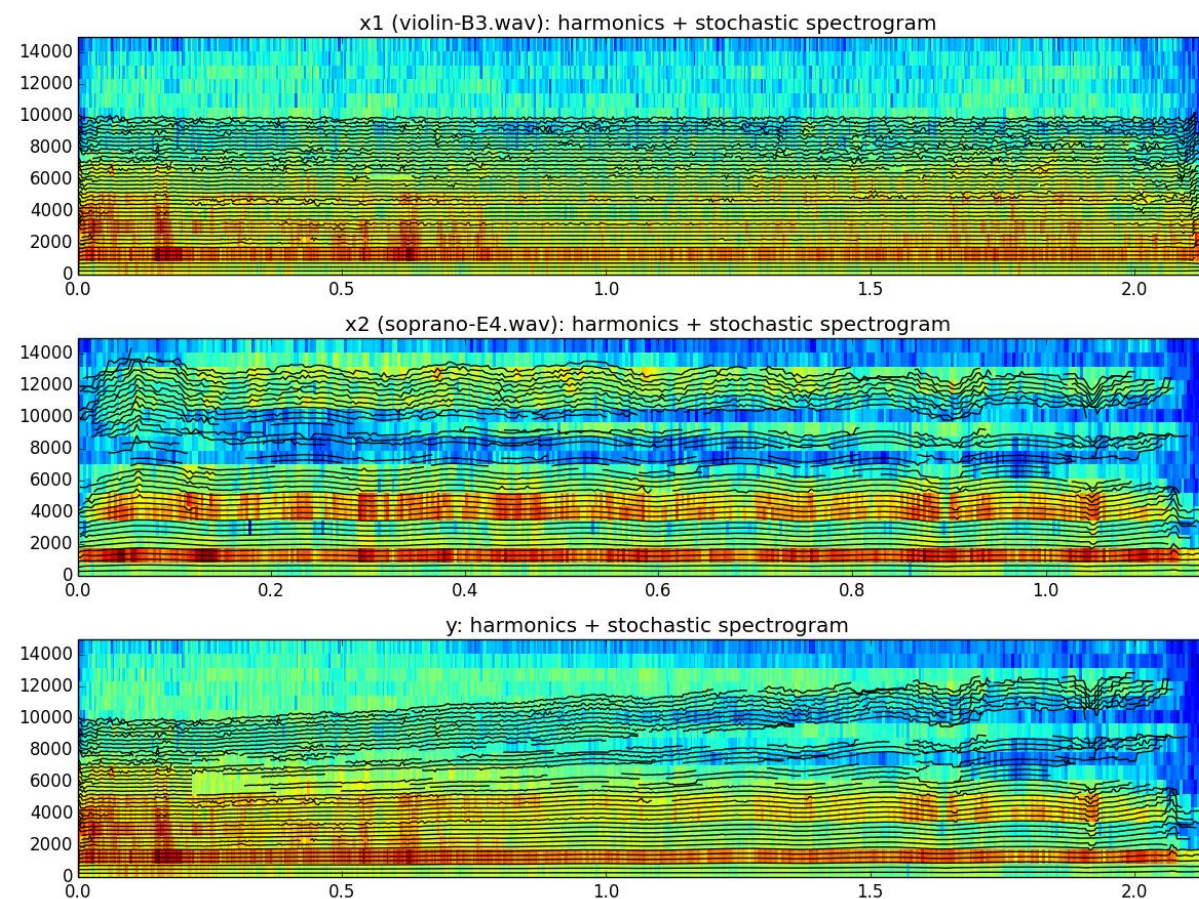
$$E'[k] = (1 - \alpha_s) E_1[k] + \alpha_s E_2[k]$$

Three interpolation factors

- α_f controls harmonic frequencies.
- α_m controls harmonic magnitudes in dB.
- α_s controls the stochastic dB envelope.
- The implementation aligns durations and matches harmonic indices.

Example: morphing a violin and a soprano

The output evolves between harmonic structures and stochastic textures



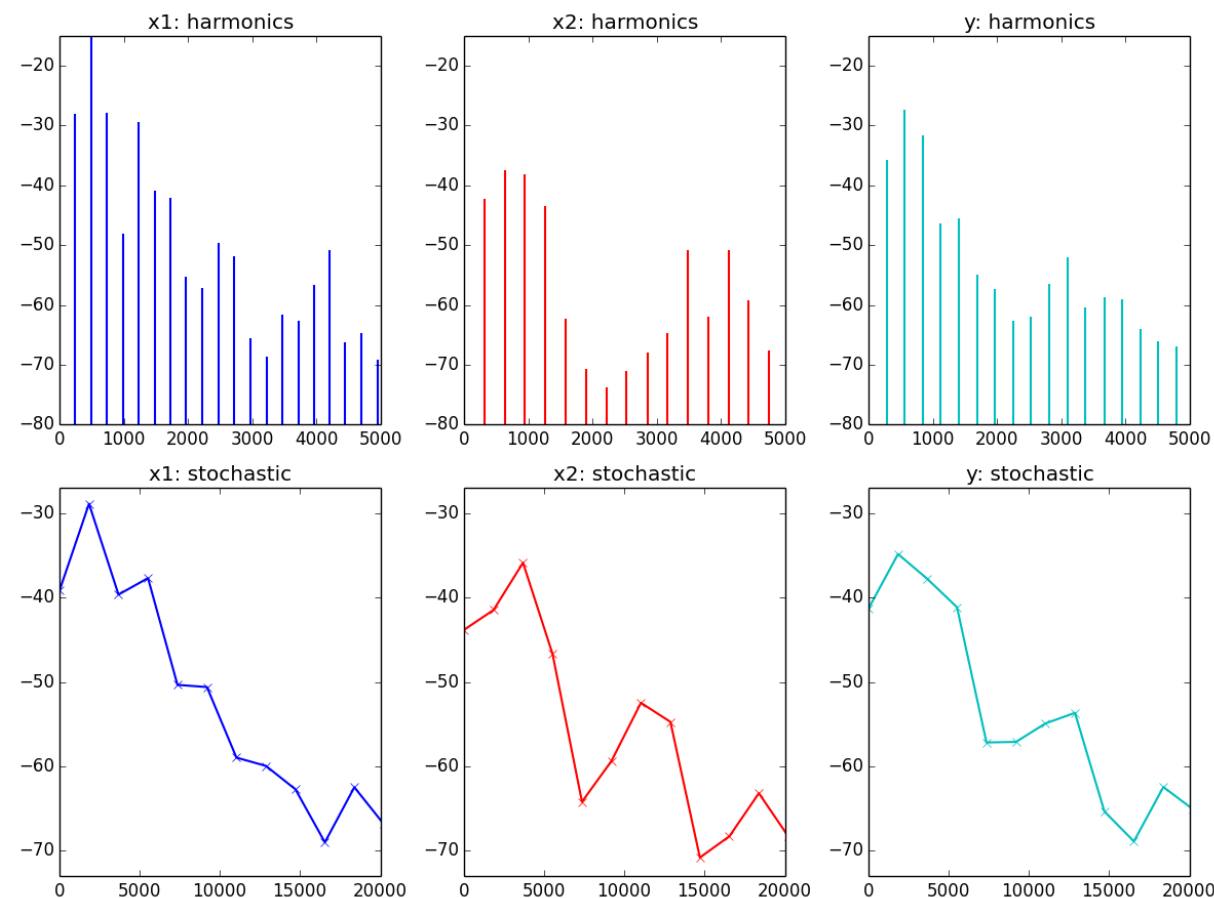
Original HPS morphing example from 8T2

How to read it

- Top: violin-B3.wav.
- Middle: soprano-E4.wav.
- Bottom: the morphed result.
- Compare relative time positions because input durations differ.
- Track motion and noise color can follow different interpolation controls.

HPS morphing at one time frame

The harmonic and stochastic parts reveal different aspects of the interpolation



Original frame comparison from 8T2

Columns and rows

- Columns: sound 1, sound 2 and output.
- Top row: harmonic magnitudes.
- Bottom row: stochastic envelopes.
- The output blends tonal and noise structure separately.

Transformation limits and audible artifacts

The representation and parameter choices determine what synthesis can preserve

What you hear	Possible cause	What to inspect
Clicks or track jumps	Discontinuous tracks or phase evolution	Track continuity, cleaning and phase synthesis
Wrong or unstable pitch	Incorrect f_0 or octave errors	$\min f_0 / \max f_0$, f_0 et and harmonic assignments
Metallic upper partials	Strong frequency stretching	Harmonic spacing and Nyquist limits
Weak or smeared attacks	Smoothing or extensive time stretching	Window length and residual/stochastic detail
Altered speech identity	Envelope moves or is poorly estimated	timbrePreservation and formant envelope

Example notebooks: where each plot comes from

The notebooks connect model arrays, transformation controls and listening

Notebook in examples/	Input example	Main concept
stft-morph-examples.ipynb	Ocean + male speech	Smoothed spectral-envelope interpolation
sine-transformations-examples.ipynb	Mridangam	Frequency and time scaling of tracks
harmonic-transformations-examples.ipynb	Vignesh singing	Scaling, stretching and envelope preservation
stochastic-transformations-examples.ipynb	Rain	Time scaling of a stochastic envelope
hps-transformations-examples.ipynb	Saxophone phrase	Coordinated harmonic and stochastic transformation

[Open the MTG transformation examples](#)

Python workflow for HPS transformations

Keep the analysis arrays visible and transform them before resynthesis

```
hfreq, hmag, hphase, stocEnv = HPS.hpsModelAnal(...)
```

```
hfreq, hmagt = HT.harmonicFreqScaling(  
    hfreq, hmag, freqScaling, freqStretching,  
    timbrePreservation, fs)
```

```
yhfreq, yhmag, ystocEnv = HPST.hpsTimeScale(  
    hfreq, hmagt, stocEnv, timeScaling)
```

```
y, yh, yst = HPS.hpsModelSynth(  
    yhfreq, yhmag, np.array([]), ystocEnv, Ns, H, fs)
```

Array meanings

- Rows: time frames.
- Harmonic columns: partials.
- Stochastic columns: envelope samples.
- An empty phase array regenerates harmonic phases.

Exercise 8: sound transformations

Two HPS tasks connect analysis choices to a perceptual transformation goal

Part 1: natural speech

- Use speech-female.wav, as in E7.
- Aim for a clear perceptual change while keeping plausible, intelligible speech.
- Analyze, check the baseline resynthesis and apply one transformation pass.

Part 2: creative transformation

- Choose a short monophonic harmonic sound from Freesound.
- The result can sound artificial. Explain the intended effect.
- Use one transformation pass and provide enough detail to reproduce it.

Open exercises/E8-Sound-transformations.ipynb and justify both stages separately.

Lab 8: analysis parameters and their consequences

Choose settings that capture the components needed for the transformation

Parameter group	Meaning	What to observe
window, M, N	Window shape, frame size, FFT size	Leakage, frequency separation and time smearing
t, minSineDur	Peak threshold and track cleaning	Faint harmonics, false peaks and short gaps
nH	Maximum harmonic count	Missing upper partials and excess weak tracks
minf0, maxf0, f0et	Pitch range and f0 acceptance	Octave errors, voiced/unvoiced regions
harmDevSlope	Harmonic frequency tolerance	Valid deviations versus incorrect assignments
stocf	Envelope size relative to spectrum	Noise detail versus compactness

Reconstruction settings and transformation settings

A faithful baseline and stable model trajectories can require different compromises

Reconstruction emphasis

- Retain spectral detail and weak components that matter perceptually.
- Check agreement with the original waveform and sound.
- Avoid explaining input structure as noise when it should be harmonic.

Transformation emphasis

- Prioritize stable harmonic identities and useful continuity.
- Keep the stochastic envelope compact enough to retune smoothly.
- Compare with E7 and explain any loss of detail that the transformation introduces.

Lab 8: transformation controls

Frequency arrays store factors. The time array stores an input–output mapping

Control	Meaning	Illustrative value
freqScaling	Multiply harmonic frequencies	[0, 1.2, 1, 1.2]
freqStretching	Harmonic-dependent stretching	[0, 1, 1, 1] keeps harmonicity
timbrePreservation	Read magnitudes from the original envelope	1 enabled, 0 disabled
timeScaling	Map input landmarks to output landmarks	[0, 0, 1, 1.4] gives 1.4× duration

These values illustrate the controls. Choose and justify values for your own perceptual goal.

Lab 8, Part 1: speech transformation

Judge pitch, speaking rate, intelligibility and spectral-envelope behavior separately

Controlled comparisons

- First listen to original speech and baseline HPS resynthesis.
- For a pitch change, compare envelope preservation off and on.
- Change one control at a time before designing the final transformation.

What to explain

- State the perceptual goal in one sentence.
- List all analysis and transformation values.
- Identify artifacts and explain a plausible cause.
- Propose a time-varying version of one control.

Lab 8, Part 2: a creative sound transformation

A reproducible design connects each parameter trajectory to a sonic effect

Input and transformation

- Use a monophonic harmonic sound of at most 5 seconds.
- Prepare mono, 44.1 kHz, 16-bit WAV and provide the Freesound URL.
- Design one transformation pass, with no mixing of independently transformed outputs.

Explanation and evidence

- Explain why the sound suits the chosen effect.
- Describe how stretching changes upper-partial spacing.
- Show how time-varying controls shape the sound.
- Discuss what the HPS approximation limits.

How to read and listen to transformation outputs

Connect every visible change to one control and one audible consequence

1 • Baseline

Compare the original and unmodified resynthesis before judging the transformation.

2 • Tracks

Check frequency movement, harmonic spacing, gaps and the Nyquist limit.

3 • Timing

Check event landmarks and compare actual time-axis values.

4 • Envelope

Separate harmonic magnitude changes from stochastic background changes.

5 • Listening

Use comparable playback level and listen to attacks, pitch stability and artifacts.

What to remember from Topic 8

Spectral and model representations provide different ways to control a sound

1 Representation

Choose the information needed for the intended transformation.

2 STFT

Filtering and envelope morphing act on windowed spectra.

3 Tracks

Frequency, magnitude and time can change independently.

4 Harmonic structure

Transposition keeps ratios; shifting and stretching can change them.

5 HPS

Retiming must align harmonics and stochastic envelopes.

6 Validation

Inspect model outputs and listen to the baseline and transformed sounds.

References and credits

Lecture materials, implementation examples and Exercise 8

- Original topic materials: Xavier Serra, 8T1 and 8T2 spectral-based sound transformations.
- Exercise: `exercises/E8-Sound-transformations.ipynb`.
- [MTG sms-tools-materials: transformation notebooks, sounds and teaching materials.](#)
- [MTG sms-tools: models and Python transformation implementations.](#)
- Serra, X. and Smith, J. O. (1990). Spectral Modeling Synthesis: A Sound Analysis/Synthesis System Based on a Deterministic Plus Stochastic Decomposition.
- Original slides: CC BY-NC-SA. Code: AGPL. Sound attribution follows the repository and Freesound source.